

Unit-I

Introduction

classmate

Date 10/07/2021

Page 4

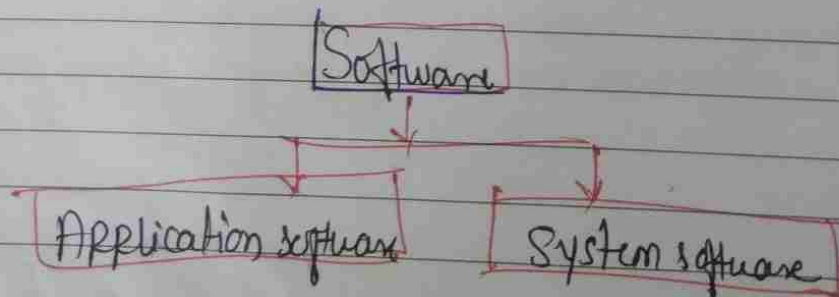
Introduction of System programming

- 1) System
- 2) Program
- 3) Programming
- 4) Software
- 5) System program
- 6) System programming

System:- A System is a group of interacting or inter-related element that act according to a set of rules.

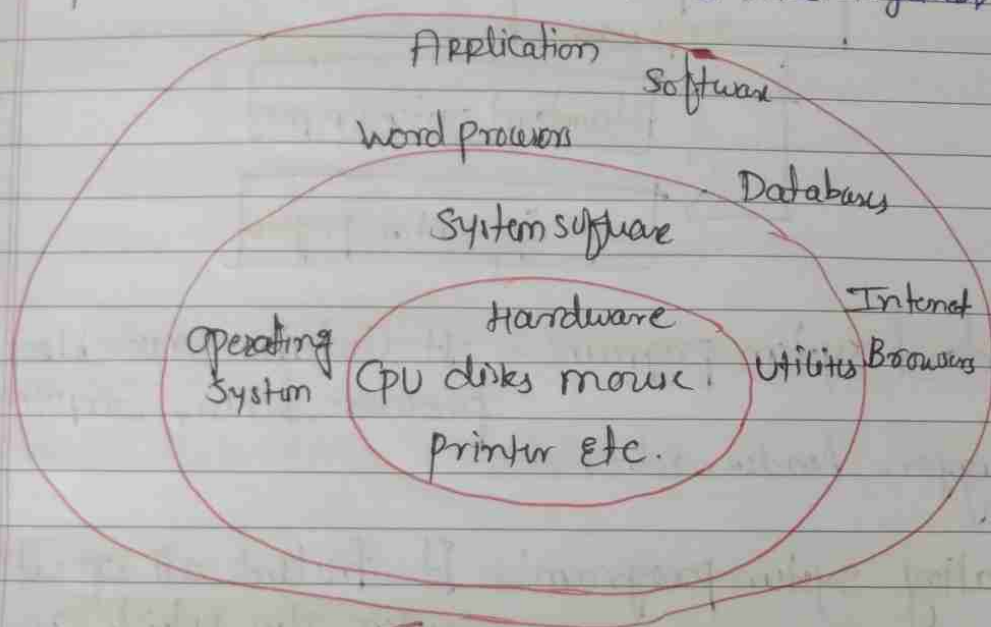
Program:- Group of Command is known as program

Software:- Software is a set of Computer Programs and associated documentation and data.



System software:- System software is a type of Computer program that is designed to run a Computer's hardware and application program.

- * system sw provides the base of infrastructure for various sw to run smoothly,
- * system sw handles the H/w i sw & n/w together.



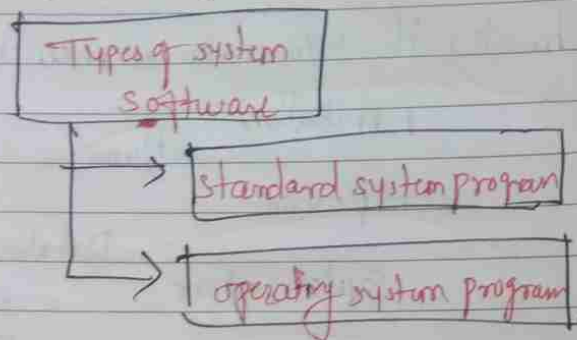
System software

- * In the other word system software can be define as the an interface between the hardware & user applications.
- * System sw Converts all human instructions written in application program into machine understanding format.
- * system sw is "OS" (Operating System). The OS can handle other program in Computer.
- * Some other Examples of system sw are compiler, Interpreter, Assembler etc.

Types of System software -

- ① standard system program
- ② operating system program

Types of System software



(1) Standard system program - It contains assembler, processor, linker, compiler, debugger, loader, editor etc.

(2) Operating system program - It includes all operating system sw which can be used to create interface between hardware and user application.

For Examples: Window, Dos, UNIX, Linux etc.

Application software - Application program is - also called as application software.

Features of application program or application software

- ① Easy to design
- ② More interactive
- ③ Close to the user
- ④ Bigger in size and requires large storage space.
- ⑤ Generally written in high level language.
- ⑥ Slow in speed.

- ⑥ Easy to manipulate and use.
- ⑦ Easy to understand.

Example -

- ① Bank management system.
- ② Database management system.
- ③ sales and purchase slw.
- ④ accounting slw.
- ⑤ word processor etc.

★ Difference between system software & Application software.

S.no	parameters	system software	Application software
1)	Definition	System slw is mainly - focused on the efficient management of the computer system.	An application program mainly focused on solving the specific problems using computer as tool.
2)	Purpose	System programs support the operation rather than any particular application.	The main focus is on the application not on the computing system.
3)	Machine-dependency	System software is - machine dependent	Application software is machine independent.
4)	Programmer-Knowledge	System programmers require knowledge about the internal computer architecture.	App ⁿ programmer requires detailed knowledge about high level lang which is used to develop an app ⁿ .
5)	portability	System program become portable using concept of bootstrapping.	APP ⁿ program becomes portable using concept of cross compiler.

- 6) Example | Example are OS, Compiler, Interpreter, assembler, etc. | Example are Microsoft, Word, Notepad, Photo Shop etc.

★ Machine Structure - Need of system programming -

★ What is system programming? Explain need of -
System programming!

Ans) System programming! - System programming is used to design and develop program which interface the computer system (hardware part) with the developer and end user.

Definition! - System program or system software is a set of programs which are developed to operate, control and extend processing capabilities of computer itself.

★ system program handles the hardware, software and network together.

★ In other words system s/w can be define as interface between the hardware and user applications.

★ system software convert all human instructions written in application program into machine understandable formats.

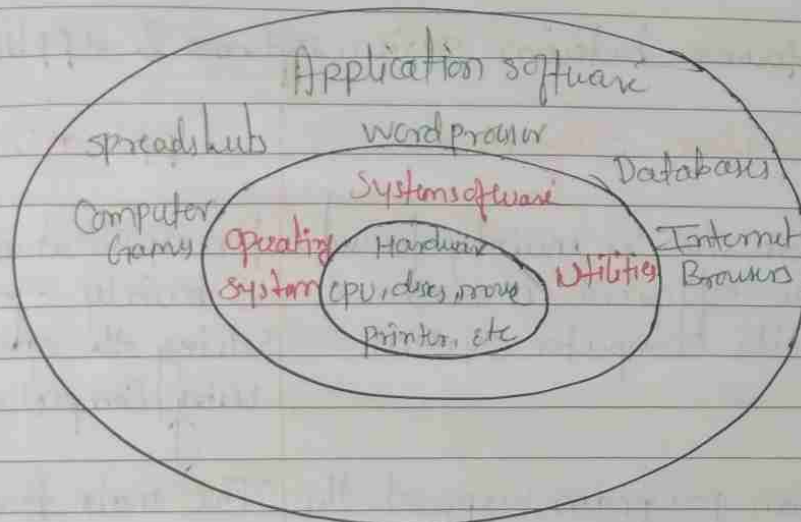
★ System software is responsible for the smooth functioning of all hardware parts and their interoperability to execute task successfully.

★ System s/w
storing
storage
etc.

★ The basic
The operating
comp

★ Some
inter

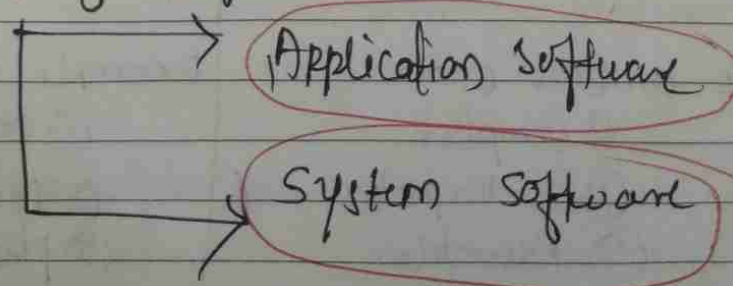
Ty



System Software

- * System SW manages basic functions of computer like - storing data in the memory, retrieving files from the storage devices, scheduling tasks as per the periods etc.
- * The best example of system software is operating system (OS). The operating system can handle other programs in computer.
- * Some other examples of system software are compilers, interpreter, Assembler etc.

Types of Software:-



★ Difference between system software & application software

And System software

Application software

Definition

Definition

Purpose

Machine dependency

Programmer knowledge

Portability

Examples

System s/w is mainly focused on the efficient management of the computer system

System program support the operation rather than any particular application

System software is machine dependent

System programmer requires knowledge about the internal computer architecture

System program becomes portable using concept of bootstrapping

Example and OS & OS

- ① Compiler
- ② Interpreter
- ③ Assembler
- etc.

An application software is mainly focused on solving the specific problem using computer as a tool.

The main focus is on the application not on the computing system.

Application s/w is machine independent

Application programmer requires detailed knowledge about high level language which is used to develop an appⁿ.

Application program becomes portable using concept of cross compiler

Example:-

- ① Microsoft access
- ② Notepad
- ③ Photoshop
- etc.

Software

Application software

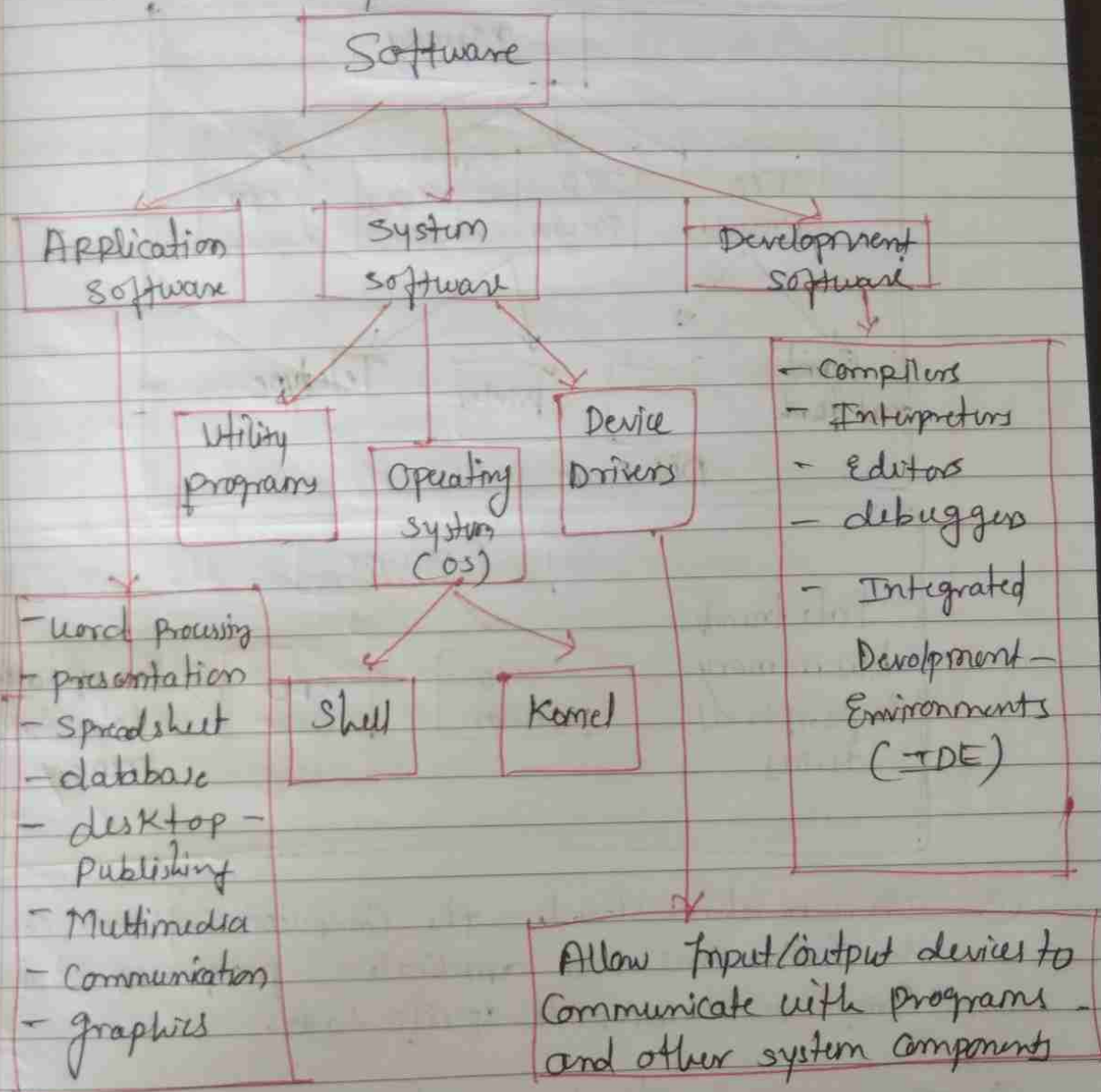
- Word p
- present
- Spread
- datab
- desk
- Publ
- Mult
- Com
- gra

★

div

- ① App
- ② Syst
- ③ Dev

Software Hierarchy :-

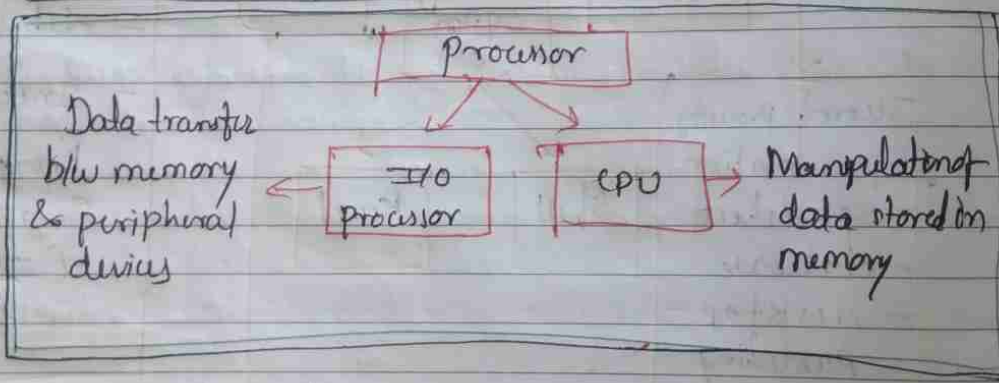
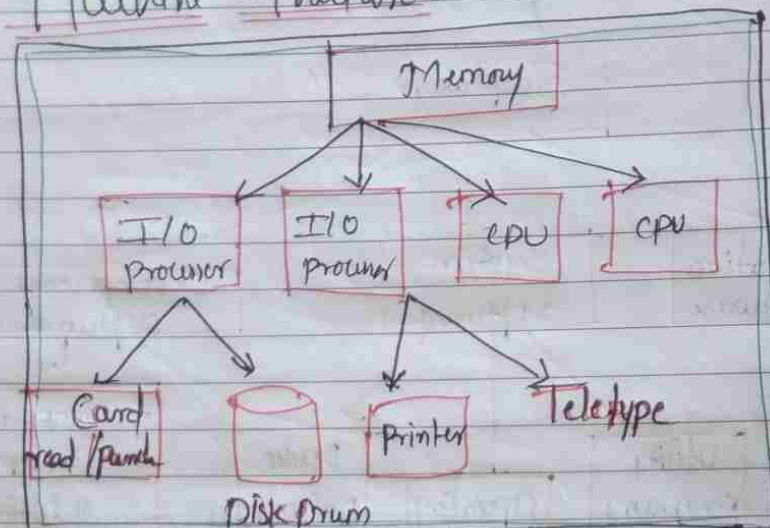


Mainly Software Hierarchy -

★ Divided into three categories:-

- ① Application Software.
- ② System Software.
- ③ Development software.

Machine Structure



① In a machine structure the Computer Software contains the instructions and application programs that allow computers to complete specific tasks.

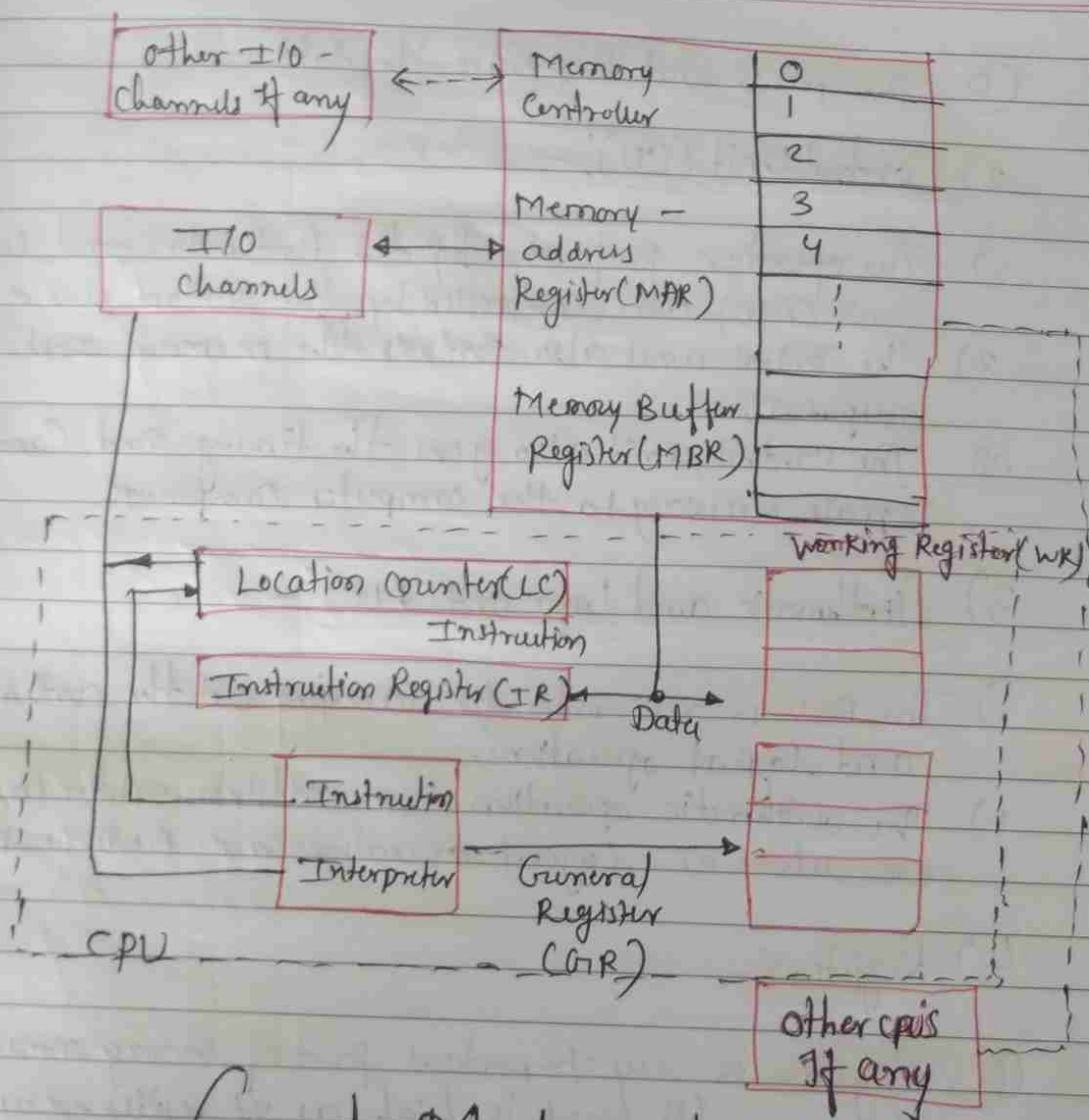
② To accomplish the tasks, following components of the computer system are very important:

P.T.O

1) Central

and a
program
CPU is
program

③ The C



General Machine Structure

- 1) Central Processing Unit (CPU): This unit which is responsible for interpreting and executing the instructions/commands of a computer program.
- 2) CPU is sometimes referred as the microprocessor or simply processor.
- 3) The CPU contains the ALU, CU and a range of registers.

① The cpu is used to process the data

2) Control Unit (CU):-

- 1) The operation performed by the Arithmetic and logic unit (ALU) are controlled by the Control unit of cpu.
- 2) The Control unit also controls the memory and input/output devices.
- 3) The Control unit also gives the timing and Control signals necessary to the computer components.

3) Arithmetic and Logic Unit (ALU):-

- 1) The ALU is responsible for carrying out the arithmetic and logical operations.
- 2) The arithmetic operation are addition, subtraction, division, etc. where as logical operations are AND, OR, NOT, etc.

4) Registers:-

- ① Register is very important part of storage area of cpu because its speed is high as it resides very close to the ALU.
- ② As a result the processing speed of ALU can be more as compared to memory location which can be anywhere in the memory.
- ③ Before going to process all the data must be stored in a Register for a quick response.

Used in the cpu.

Some of the registers-

1) MAR

2) MDR

3) AC

4) PC

5) CI

Registers used in the CPU

1)	MAR	Memory Address Register	It preserves the memory location of data that is desired to be accessed.
2)	MDR	Memory Data Register	It preserves data that is being transferred to or from memory.
3)	AC	Accumulator	The accumulator is used to store the intermediate arithmetic and logical results.
4)	PC	Program Counter	It is used to store the address of the next instruction to be executed.
5)	CIR	Current Instruction Register	It is used to store the current instruction during processing.

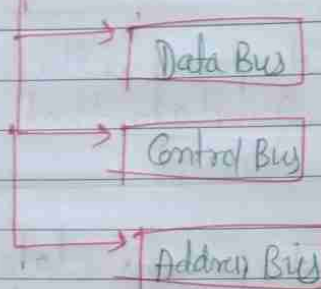
There are three types of buses linked among the CPU primary storage and the other devices in the computer system. They are:-

Bus types:- ① Data Bus

② Control Bus

③ Address Bus

Bus Types



1) Data Bus -

Data in the the data bus can be transferred in both directions. i.e. bi-directional.

2) Control Bus -

This bus is responsible for transmitting the signals which finds a given address in primary storage representing where the data should be placed.

3) Address Bus -

1) The bus is responsible for transmitting the signals which indicate whether to read or write data to or from primary storage address, input device or output device.

② The Internal bus is also called as Internal-data bus.

③ Memory bus or local bus as it is proposed to -

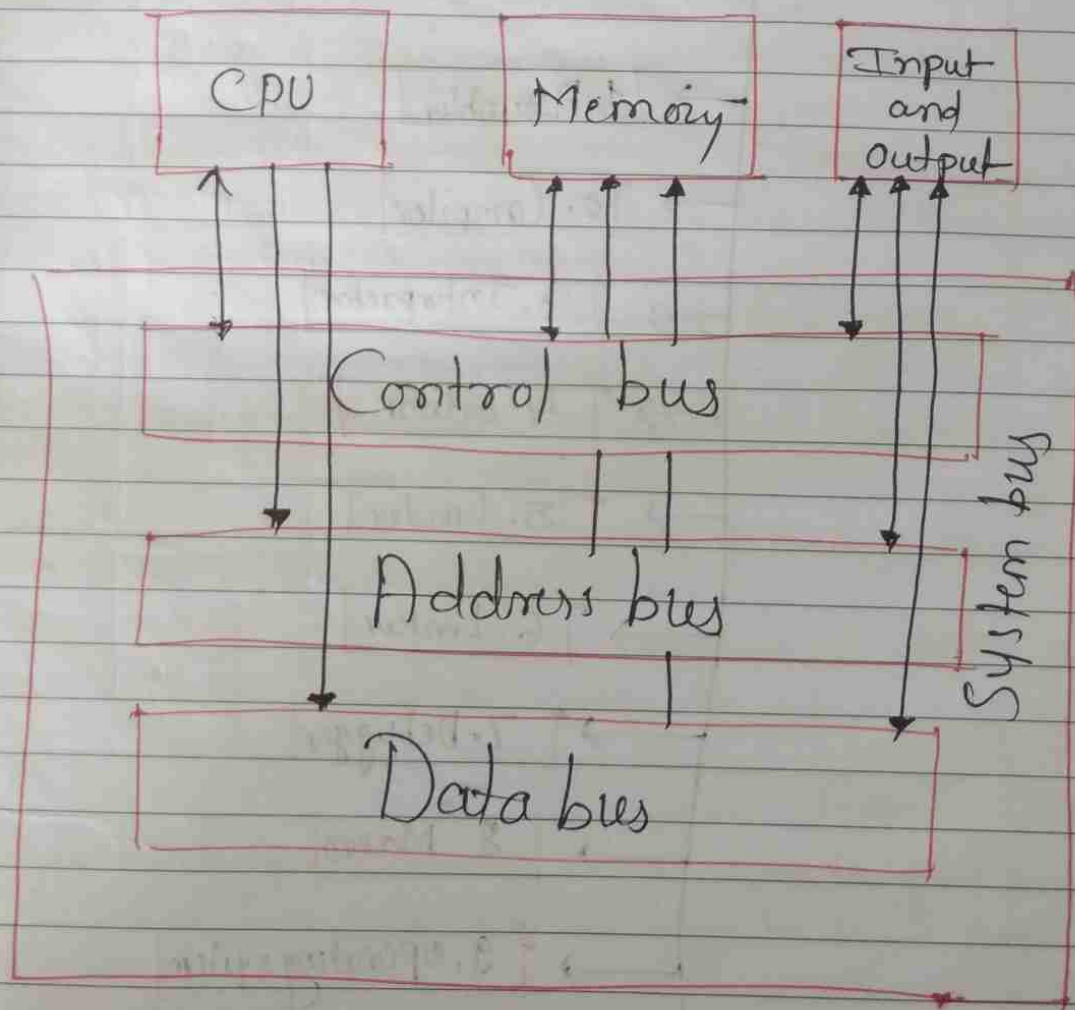
local

④ Elec
bus
pro



local devices.

- ⑨ Electronic pathways are used to make an external bus that connect the various external devices like projector scanner etc to the computer.



Block diagram of bus

★ Explain Components of System programming

Evolution of Components of system programming

The component of system software -

Components of system software

- 1. Assembler
- 2. Compiler
- 3. Interpreter
- 4. Editor
- 5. Loader
- 6. Linker
- 7. Debugger
- 8. Macro
- 9. operating system
- 10. Device driver

Components of system software

(1) Assembler: A program written in Assembler is system software which convert programs written in assembly language into machine language.

(2) Assembler verify every instruction for accuracy and produce diagnostic message when errors are encountered.

(2) Compiler: - Compiler is a system program that translate High level language into low level language.

Source code → Compiler → Object code

(3) Interpreter: - Interpreter is also a translator like Compiler but Interpreter takes one line at a time and translate it into object code.

Source code → Interpreter → Object code

(4) Editor: - Editor are editing the text, traversing through the text, viewing and displaying the text etc.

Types of Editors

Word processor

Line Editor

Screen Editor

Stream Editor

Strutain Editor

(5) Loader: - Loader perform the function of placing Object code into main memory for execution purpose.

Object code → Loader → Executable code

(6) Linker: - Linker is used to modify the addressess of notations according to new memory location.

Object code → Linker → Executable code

(7) Debugger: - Debugger is Computer program used to find out errors which are also called as bugs in source program.

(8) Macro: - Macro is defined as a group of Instructions which can be replaced when it is called by macroprocessor. Macro - used for fast execution.

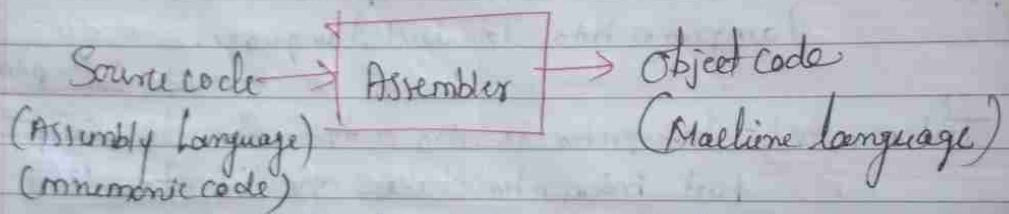
ALP with macro → Macroprocessor → ALP with macro

(9) Operating system: - OS is defined as system program that enables users to commⁿ with hardware.

(10) Device driver: - Device driver is a system program used to control number of devices which are attached to Computer.

Assembler

- * Assembler is a software that converts an assembly language code to machine code.
- * Assembler verifies every instruction for accuracy and produces diagnostic message when errors are encountered.



The translator is called assembler and the Source language is called an assembly language.

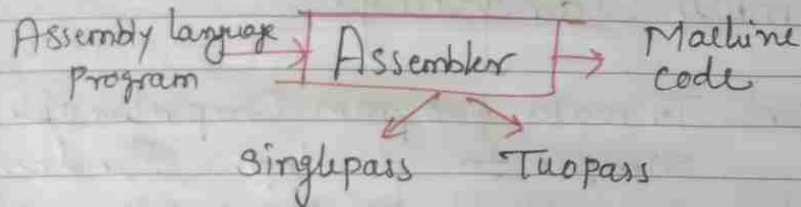
Assembler function:-

- 1) Translate mnemonic opcodes to machine language.
- 2) Convert symbolic operands to their machine addresses.
- 3) Build machine instructions in the proper format.
- 4) Convert data constants into machine representation.
- 5) Error checking is provided.
- 6) Changes can be quickly and easily incorporated with assembly.
- 7) Variables are not represented by symbolic names, not as memory locations.

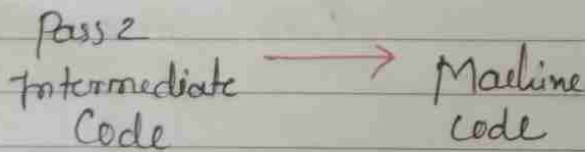
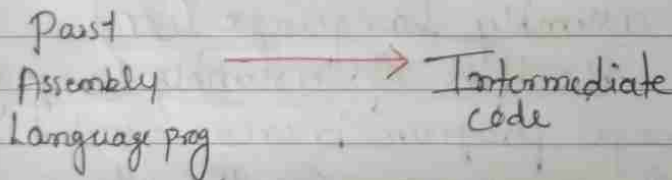
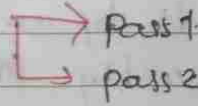
Assembler mainly two types:-

- 1) Single pass Assembler.
- 2) Two pass Assembler.

★ Two pass Assembler :-



Two pass Assembler:-



4 Data structure -

- 1) MOT
- 2) Symbol Table
- 3) Literal Table
- 4) Pool Table

Note:-

Assembler Definition:- An assembler is defined as a language processor which translates the assembly language program into a machine language program.

Single pass Assembler

Assembly language:-

Definition:- To make program Comprehensible and Simple to develop, the language instructions are replaced by mnemonic code and symbolic operands by a language. Such language is known as assembly language.

Advantages of assembly language (or)

Benefits of assembly language

- ① Assembly language program increases readability with the help of mnemonic operation code.
- ② It is memory efficient.
- ③ It is not required to keep track of memory locations.
- ④ Assembly language is faster in speed.
- ⑤ Easy to make Insertions and deletions.
- ⑥ Assembly language known as is hardware oriented.
- ⑦ Assembly language is useful in Embedded system as it have less resources, it helps in managing size and code.
- ⑧ Require less instruction to compute the same result.

⑤ It can a
Can be

Disadv

① long pr
on sm

② It tak

③ It is

④ Assum

⑤ Assum
parting

⑥ Ass
as +
level

★ Ele

fe

① Mh

② S

③ D

- ⑤ It can access hardware driver and system code easily which can be difficult using high level code.

Disadvantages of Assembly language:-

- ① Long programs written in such language cannot be executed on small size computers.
- ② It takes lot of time to code or write the program.
- ③ It is more complex in nature.
- ④ Assembly language is machine dependent.
- ⑤ Assembly language code is platform dependent. so - porting to another platform is not easy.
- ⑥ Assembly code is more difficult to debug and validate as there are more chances of errors than in high-level code.

★ Elements of Assembly Language programming

OR

Features of Assembly language

- ① Mnemonic operation code
- ② Symbolic Operands
- ③ Data declaration

Elements / Features of Assembly language

→ Mnemonic operation code

→ Symbolic operands

→ Data declaration

Elements of Assembly Language programming

1) Mnemonic operation code:-

① Assembly language uses mnemonic operation codes for machine instructions rather than numeric operation code.

② Numeric codes are difficult to memorize whereas mnemonic codes are easy to understand as well as easy to memorize.

Example:- we can use mnemonic code ADD instead of using numeric code 01 to indicate instruction of addition.

2) Symbolic operands:-

① Assembly language makes use of symbolic names to denote data or address.

② The programmer's job can be reduced by binding to

Installing python Bootstrap

- 1) In these names.
- 2) This can be done with help of assembler.

Example! - we can specify symbols like ADDITION, SUBTRACTION, etc. to store data rather than specifying memory location.

3) Data Declaration! -

- 1) The assembly language data can be declared in range of notations.
- 2) The assembler can help to convert constants to it's - machine representation.

Example! - The constant data can be declared in decimal notation instead of machine representation.

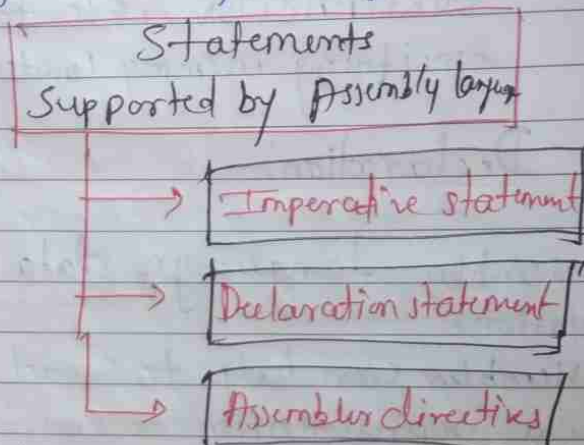
Statement format! - Assembly language format
[Label] <opcode> <operand specification> [, <operand specification> , ...]

Explain simple assembly language! - In assembly language - Every statement is composed of two operand and opcode having the following format.

[Label] <opcode> <operand 1> <operand 2>

Assembly Language statements!

There are three types of statements that can be supported by assembly language they are:-



Statements supported by assembly language.

1) Imperative statement! — Imperative statement specifies an action to be taken when execution of assembled program is taking place.

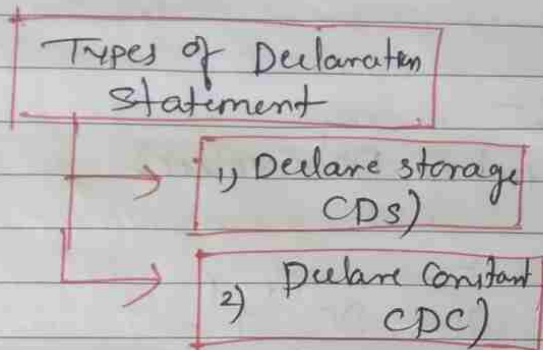
Each assembler imperative statement is translated into machine code through assembler for further processing.

Example! — SUB, BREG, X

Here the above statement states that subtract the content of X from the content of BREG.

2) Declarative Statement! — Declarative statement used to declare variables and constants. There are two types of

declaration statement-



Types of Declaration Statement

▷ Declare storage (DS):- The DS statement associate names with the reserved memory, The DS statement has following format:-

Syntax:-

<symbol> DS <constant>

Example:-

1. ONE DS 1
2. TWO DS 400
3. TEN DS 10

- 1) In the first example, the 1 word memory location is reserved and it's associate name with it.
- 2) In the second statement 400 words memory location are reserved and it's associate name Two with it.
- 3) In third statement 10 words memory location are reserved and it's associate name TEN with it.

2) Declare Constant (DC):- The DC statement is used to create the memory words

holding constants. The DC Statement has following format -

Syntax:-

<symbol> DC <value>

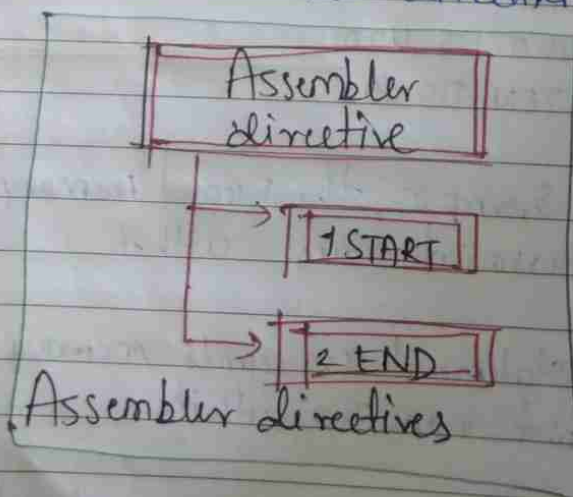
Example:-

ONE DC '1'

In above Example the memory location containing the value 1 and its associates name ONE with it.

★ Assembler Directives:-

An Assembler directive guide the assembler to perform certain action during the assembly of program. There are two assembler directives:-



1) START:-

Syntax:-

START <constant>

This directive indicate that the first word generated by assembler is stored in memory location -

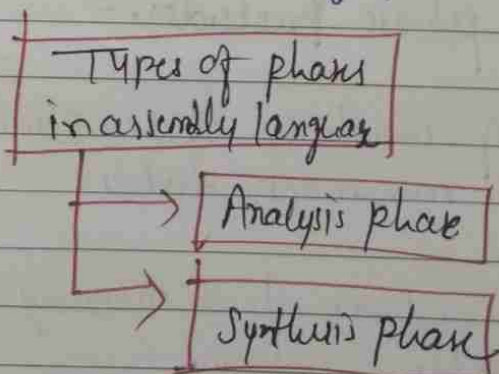
specified by <constant>

Example: START 400
END

★ Assembly Scheme:- To build up a design specification of an assembler, four step method is used. These 4 steps are:-

- ① Isolate the information that needed to execute task.
- ② Design data structure to record the information.
- ③ Decide the process with which is used to take and preserve the information.
- ④ Decide the process to execute the task.

Overview of assembly process:- There are two types of phases in assembly process:-



Types of phases in Assembly process

Analysis phase:- Analysis phase used to determine the meaning of given statement. The most important task of analysis phase is to build the system symbol table for the synthesis phase to proceed.

- * Symbol table contains symbolic name and their address. The main memory addresses are associated with the symbolic name that is called as memory allocation.
- * To implement memory location a new data structure is introduced called as location counter.

Synthesis Phase:-

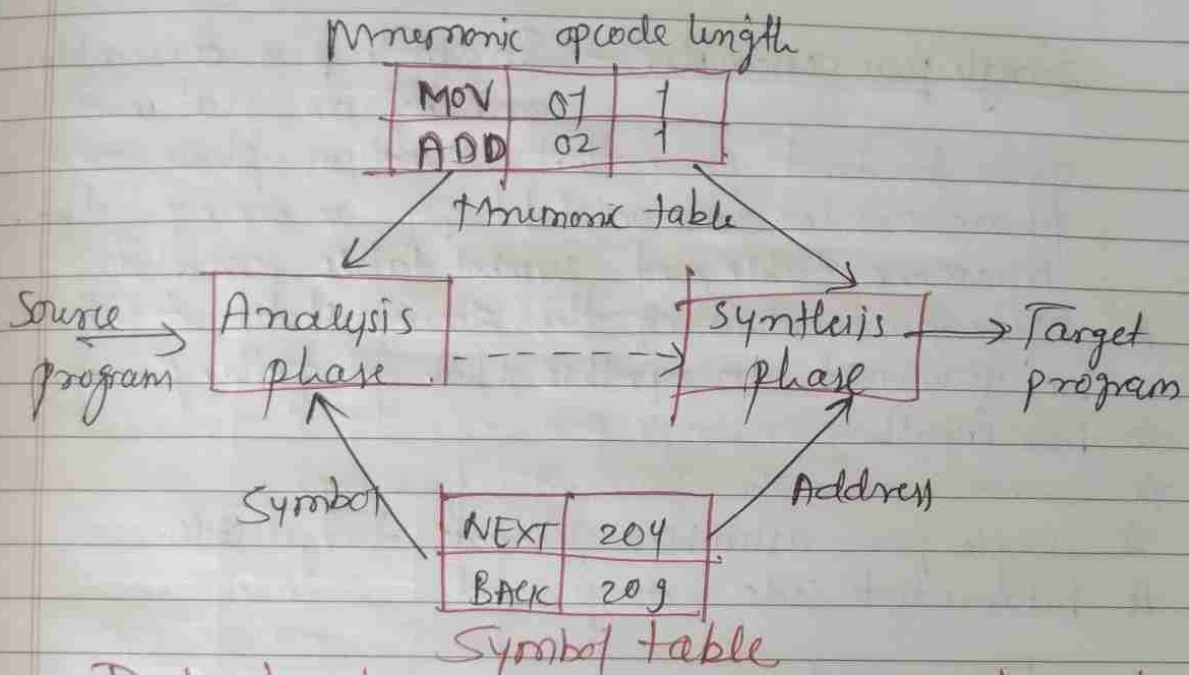
Synthesis phase used to build the machine instruction for every assembly statement. It uses the symbol table which is generated by analysis phase during analysis of statements and mnemonic table.

Synthesis phase includes:-

- ① Search of machine code equivalent to the mnemonic from the mnemonics table.
- ② Search of address of a memory operand from the symbol table.
- ③ Synthesize a machine instruction.

assembler

Show the two phases of an

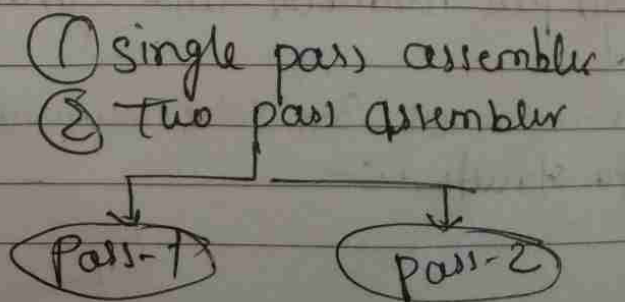


Data structure used in analysis and synthesis phase

★ Pass of Assembler:-

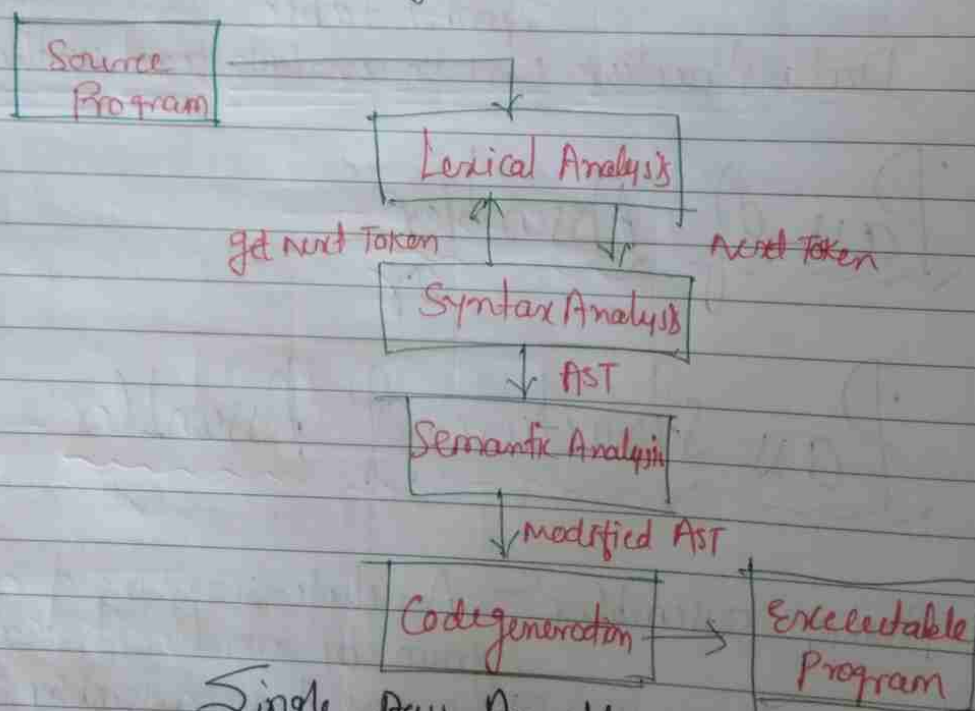
Pass structure of Assembler:-

Pass of assembler:- A whole scanning of a source program by assembler is called as a pass of assembler. There are two assembly schemes such as single pass assembler and two pass assembler.



Single pass assembler:- In single pass assembler, one by one statement is given to first phase that is analysis phase. In this phase opcodes are validated by concerning the mnemonic table and symbol table created.

- ★ After the analysis of this source statement, it can be forwarded to synthesis phase for further processing.
- ★ Does everything in single pass.
- ★ Cannot resolve the forward referencing.
- ★ Single pass assembler generate target code.
- ★ Intermediate code not generated.



Single Pass Assembler

Single pass Assembler with example:- With the help of data structure we can solve numerical problem.

Data structure:-

Data Structure

- ① MOT
- ② symbol Table
- ③ Literal Table
- ④ pool Table
- ⑤ TIT (Table of Incomplete Instruction)

★) ~~Ques~~ → Consider the following program & generate target code.

Ex ⇒ Source Programme

Target code

```
START 100
MOVER AREG, A
PRINT B
ADD BREG, = '9'
SUB BREG, D
COMP CREG, = '23'
LTORG
-
ADS 3
LABEL EQU A
ORIGIN 500
L1: MULTCREG, = '7'
DD 8
END
```

Solⁿ → for solve the given program, Data Structure used - as follows:-

- ① MOT
- ② symbol Table
- ③ Literal Table
- ④ pool Table
- ⑤ TIT

(MOT)		(POT) #	
op-code	Mnemonic	Op-code	Mnemonic
01	MOVER	01	START
02	MOVEM	02	END
03	ADD	03	EQU
04	SUB	04	ORIGIN
05	MULT	05	LTORG
06	DIV		
07	BC		
08	COMP		
09	PRINT		
10	READ		

(DL)	
Op-code	Mnemonic
01	DSI
02	DC

REGISTER	
01	A REG
02	B REG
03	C REG
04	D REG

Given Source Program - Target program

START 100
 MOVER AREG, A
 PRINT B
 ADD BREG, =19
 SUB BREG, D
 COMP CREG, =23
 LTORG
 ADS 3
 LABEL: EQU A

Find
 Target
 Code

ORIGIN 500

L1: MULT CREG₁ = '7'

D DC 8

END

Find

Target code.

Source Programme

Target code

START 100

MOVER AREG₁, A

PRINT B

ADD BREG₁ = '9'

SUB BREG₁, D

COMP CREG₁ = '23'

LTORG

-

ADS 3

LABEL: EQUA

ORIGIN 500

L1: MULT CREG₁ = '7'

B DC 10

MOVEM CREG₁ = '7'

D DC 8

END

01 - 100

100) 01, 01

101) 09 -

102) 03 02

103) 04 02

104) 08 03

105) - - 009

106) - - 023

107) - - 003

No code generation

No code generation

500) 05 03

501) - - 010

502) 02 03 008

503) - - 008

504) - - 007

Table of Incomplete Instructions (TII)

LC NO.

Incomplete Instruction

100

A

101

B

102

= '9'

103

D

104

= '23'

500

= '7'

502

= '7'

Symbol Table		
Sym-no	Symbol	Address
01	A	107
02	B	501
03	D	503
04	LABEL	107
05	L1	500

Literal Table			POOL-TAB
Lit-no	Literal	Address	
01	=19'	105	01
02	=123'	106	02
03	=17'		

Source programme	Target Code
START	01-100
MOVER AREG ₁ , A	100) 01 01 107
PRINT B	101) 02 - 501
ADD BREG ₁ = 19'	102) 03 02 105
SUB BREG ₁ , D	103) 04 02 503
COMP CREG ₁ = 123'	104) 05 03 106
LTORG	105) - - 009
-	106) - - 023
ADS 3	107) - - 003
LABEL: EQU A	No code generation
ORIGIN 500	No code generation
L1: MULT CREG ₁ = 17'	500) 05 03 504
B DC 10	501) - - 010
MOVEM CREG ₁ = 17'	502) 02 03 504
D DC 8	503) - - 008
END	504) - - 007

★ Pass Structure of Assembler:-

Two Pass Assembler:-

- ① In two pass assembler the forward reference problem can be resolved easily because in first pass all the processing is done and all symbols that are used in the source program has the valid memory addresses associated with them.
- ② So the second pass only use the symbol table and generates the target code by using the address found in symbol tables.
- ③ Following are the tasks that are performed during of two pass of assembler.

Pass-I :- Pass-I the analysis phase will start scanning the program. PC will be initialized to 100.

Consider the following example:-

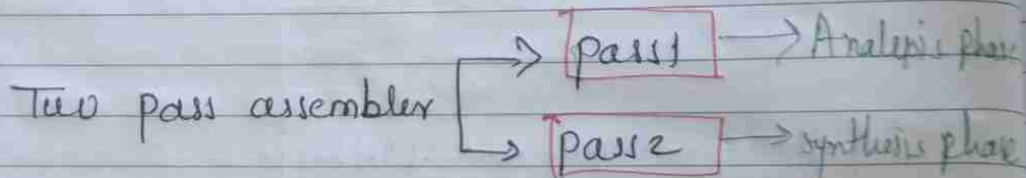
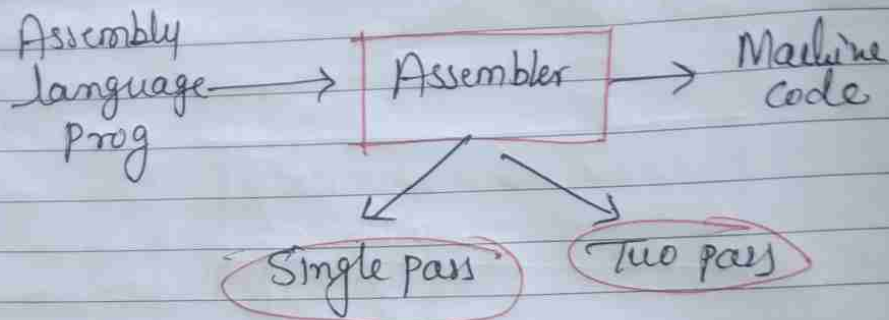
```
START 100
MOVER AREG, ONE
MUL BREG, '5'
ONE DC '7'
END
```

where -

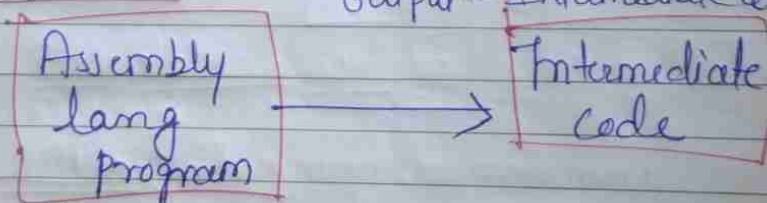
The instruction MOVER, AREG, ONE will be converted to machine instruction - 46 - 7 - ONE.

The pass I of 2-pass assembler is meant for defining the symbols, recognizing the mnemonic opcode and register. The flow

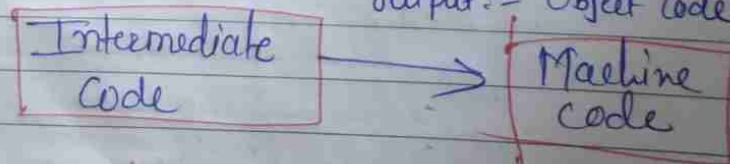
Part 1: - Two pass Assembler



① pass 1 : — Input: - source program
output: - Intermediate code



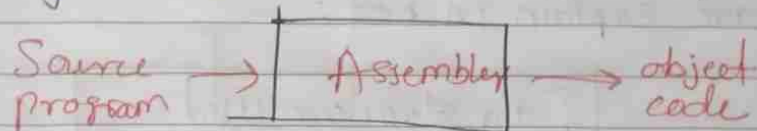
② Pass 2 : — Input: - Intermediate code
output: - Object code



4 Data structures used in pass-1 : —

- ① MOT
- ② Symbol Table
- ③ Literal Table
- ④ Pool Table

Assembler:- Assembler is a language processor that convert an assembly language program into machine language program.



Assembler mainly Two Types:-

- Single pass
- Two pass

Two pass Assembler:-

Need of 2nd pass in Assembler:- The no. of times a program or its equivalent is scanned during the language processing activity is known as pass of language processor.

Design of two pass Assembler:-

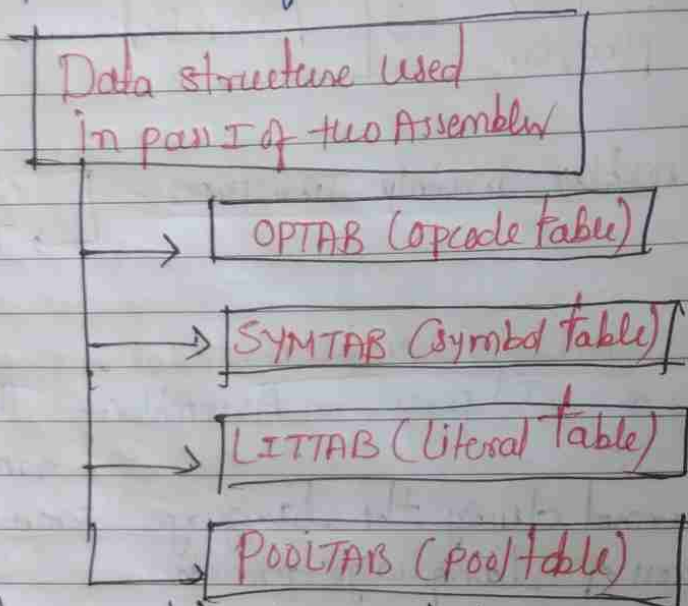
Pass 1:- (Analysis of source program)

- 1) Separate the Symbol, mnemonic, opcode and operand fields.
- 2) Build the Symbol table.
- 3) Perform LC processing.
- 4) Construct Intermediate representation.

Data structure of used in pass-I assembler
to implement two pass assembler

Pass of The Assembler

Pass I of two pass assembler uses some data structures. They are explain in brief:



Data structure used in pass I of two pass assembler

<1> OPTAB - It is used to mnemonic operation code and convert them to their equivalent machine language. In more difficult assemblers the table also contain information about instruction format and length.

- 1) In pass 1 the OPTAB checks the operation code in the program and validates it.
- 2) OPTAB also called as mnemonic Table. The mnemonic table contain three fields.

- ① opcode
- ② class
- ③ Info.

① Class - Where class indicate the type of statement of given opcode.

- 2) Fields - The information fields (info) fields contain the pair (opcode, length)
- 3) opcode - where class indicates the type of statement of given opcode.

<2> SYMTAB (Symbol Table)

- 1) Symbol table include the name and address of each label in the source program.
- 2) In pass 1 all the symbols used in source program are added in the symbol table with the corresponding address and length.
- 3) In pass 2 addresses of all the symbol used in program are taken from symbol table and added into the assembly instructions to generate target program.

<3> LITTAB (Literal Table)

Literal table include the name of literal and their corresponding address from the source program.

<4> POOLTAB (Pool Table)

Pool table is used to record the pools in the literal table.

They all are data structures used in pass I of two pass assembler

Algorithm for pass-I of two pass Assembler.

⟨1⟩ Initialize LC=0
Initialize Current-pool=f
Initialize Lit-entry=1

⟨2⟩ Perform following steps

While next statement is not an END statement.

(i) Isolate label, opcode, operand from statement.

(ii) If label is present then add label into symbol table. Enter pair(label, LC) into SYMTAB.

(iii) If opcode is LITRG statement then

a) Assign addresses to all literals from LITTAB
LITTAB [Current-pool] to
LITTAB [Lit-entry-1]

b) update the LC.

Current-pool = Current-pool + 1;

c) Generate Intermediate code.

(iv) If opcode is START or ORIGIN statement then.

a) LC = value specified in the operation field.

b) Generate Intermediate code

(v) If opcode is EQU statement then.

a) Address of symbol is now the address specified in operand field.

b) Generate Intermediate code.

(vi) If opcode is an declaration statement then

- a) perform routine Id specified in optab.
- b) $LC = LC + \text{length from optab}$
- c) If operand is a literal then.

(vii) Add literal into LITTAB (Lit-entry)

(ix) Lit-entry = Lit-entry + 1;

(x) Generate Intermediate Code.

a) If operand is a symbol then.

b) Add symbol into SYMTAB.

c) Generate intermediate code (s, code) (s, symbol/entry no)

<3> If END statement then

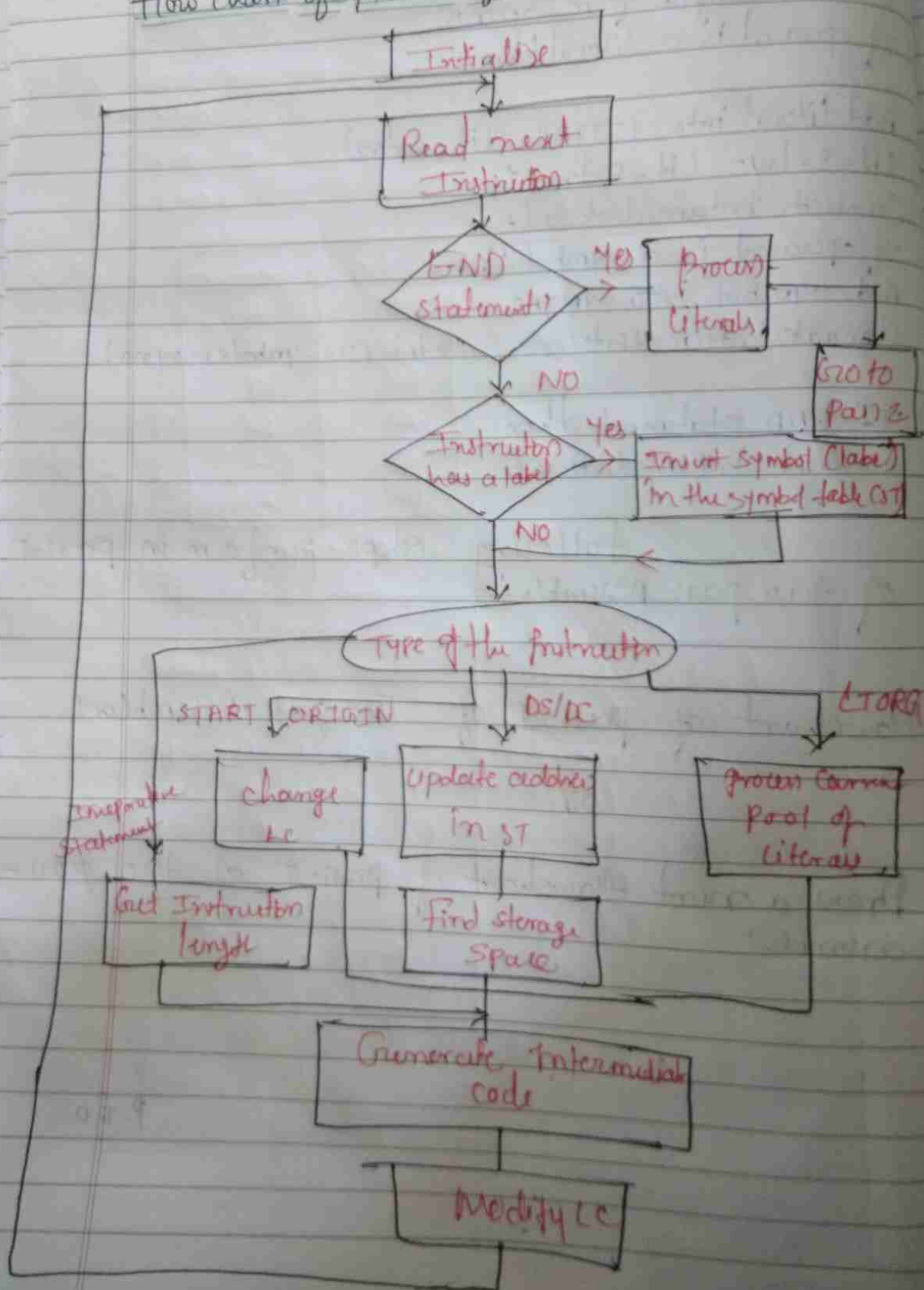
a) Generate intermediate code.

Following steps perform in pass-I of two pass Assembler.

★ Flowchart of pass-I of Two pass Assembler
or

Draw a neat flowchart of pass-I of two pass-assembler!

Flow chart of pass-I of Two pass Assembler:-



Pass 1

- ① Same p
- ② For ke
- ③ is used.
- ④ Moving
- ⑤ Symbol
- ⑥ Am.
- ⑦ Pseudo
- ⑧ symb
- ⑨ each
- ⑩ Literal
- ⑪ (LT)
- ⑫ Label
- ⑬ Table
- ⑭ A co
- ⑮ proc
- ⑯ de

In

Shed
gen

STP

A

Li

AD

M

D

L

Pass 1: Relocation

- ① Source program is used as input.
- ② For keeping track of each instruction, location counter (LC) is used.
- ③ Machine operation Table (MOT) is used to indicate the symbolic mnemonic for each instruction with its length. An instruction length is 2, 4 or 6 bytes.
- ④ Pseudo operation Table (POT) is used to represent the symbolic mnemonic and also action to be taken for each pseudo op code used in pass 1.
- ⑤ Literals of the program is stored in the Literal Table (LT) with their assigned location.
- ⑥ Labels and its corresponding value is stored in the symbol-table.
- ⑦ A copy of the input to be used later by pass 2 for processing. Normally it is stored in the secondary storage device.

Intermediate code form:-

Show content of symbol table at the end of pass I and - generate intermediate code.

START 100

A DS 3

L1 MOVER AREG, B

ADD AREG, C

MOVEM, AREG, D

D EQU AT-1

L4 PRINT D

ORIGIN A-1

C DC '5'

ORIGIN L₂+4

STOP

B DC '19'

END L₁

Solⁿ ⇒

Symbol	Address
A	100
L ₁	103
D	101
L ₂	105
C	99
B	109

Symbol Table

Variant I	Variant II
(AD, 01) (C, 100)	(AD, 01) (C, 100)
(DL, 02) (C, 3)	(DL, 02) (C, 3)
(TS, 04) (I) (S, 03)	(TS, 04) (I), B
(IS, 01) (I) (S, 04)	(TS, 01) (I), C
(IS, 05) (I) (S, 05)	(TS, 05) (I), D
(AD, 04) (S, 01)	(AD, 01) (S, 01)
(TS, 10) (OS, 05)	(IS, 10), D
(AD, 03) (S, 01+1)	(AD, 03) A+1
(DL, 01) (C, 5)	(DL, 01) (C, 5)
(AD, 03) (S, 06)	(AD, 03) L ₂ +4
(IS, 00)	(IS, 00)
(DL, 01) (C, 15)	(DL, 01) (C, 15)
(AD, 02) (S, 02)	(AD, 02) L ₁

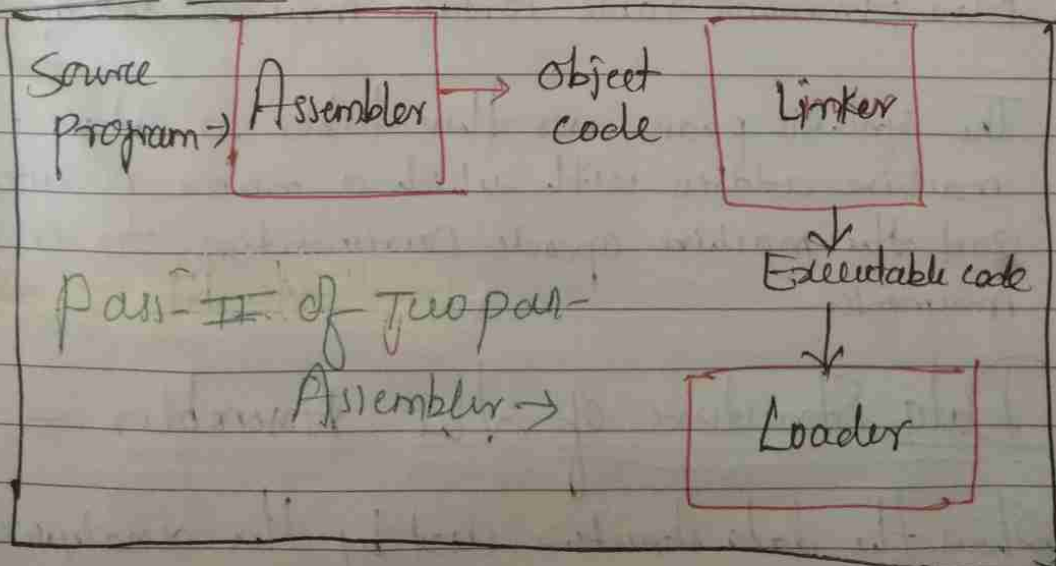
- * Intermed fields
- * To pres
- * Inter pair
- * Follow classes

- * Intermediate code is defined as group of IC units. Include three fields such as address, opcode and operand.
- * To present Intermediate code two variants are used.
- * Intermediate code uses mnemonic field which preserve the pair such as class of statement, code)
- * Following tables shows the numeric codes for different statement classes.

Mnemonic opcode	Machine code
DC	01
DS	02

Mnemonic opcode	Machine code
START	01
END	02
ORIGIN	03
EQU	04
LTORG	05

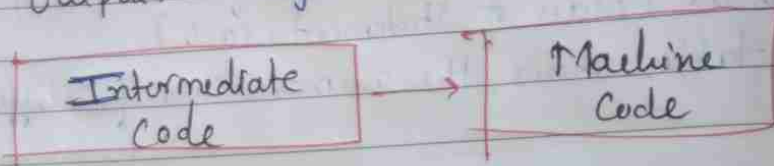
Pass-II of the Assembler



Pass II - process the intermediate representation for synthesis - the target program.

Input:- Intermediate code

Output:- Object code



Explain different data structures used to design pass I and pass II of assemblers with examples.

Data Structure used in synthesis phase -

① Symbol table

② Mnemonics table

① Symbol table contains name & address field. Symbol table is built by the analysis phase. It also contains flag to indicate errors.

② Mnemonic table contains mnemonic and opcode.

③ The synthesis phase uses these tables to obtain the machine address with which a name is associated and the machine opcode corresponding to a mnemonic.

Data Structure of the Assembler:-

Shows the data structure used by the analysis and

Synthesis phase task:-

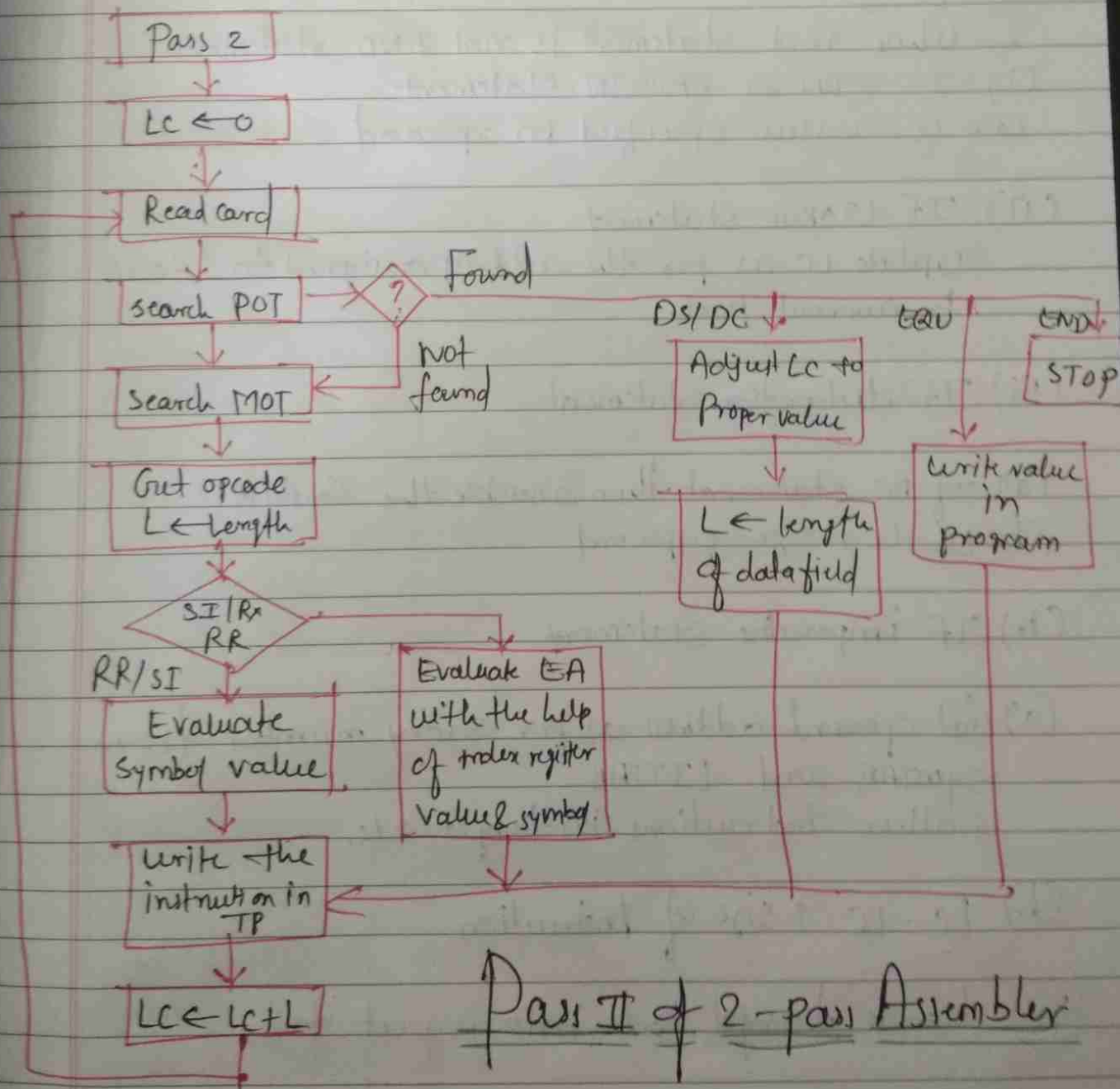
- ① Read the machine opcode corresponding to the mnemonic from the mnemonics table.
- ② Get an address of a memory operand from the symbol table.
- ③ Synthesize a machine instruction.

Pass 2 database:-

- ① Copy of source program input to pass 1.
- ② Location counter.
- ③ MOT represents the following for each instruction:-
 - a) Symbolic mnemonic
 - b) Length
 - c) Binary machine opcode
 - d) Format
- ④ POT is used for each pseudo-op the symbolic mnemonic and the action to be taken in pass 2.
- ⑤ Pass 1 prepared the symbol table which contains labels and its corresponding value.
- ⑥ Base Table indicates which registers are currently specified as base registers by using pseudo-ops.
- ⑦ A work space is used to hold each instruction as it

Various parts are being assembled together.

- ⑧ A work space (PRINT LINE) used to produce a printing listing.
- ⑨ A work space (PUNCH CARD) also used for outputting.
- ⑩ An output deck of assembled instructions in the format needed by the loader.



Pass II of 2-pass Assembler

Pass II of Assembler:-

Pass I contains an Intermediate code, SYMTAB and LITAB, Pass II uses these data structures and converts the intermediate code into target code with the help of SYMTAB and LITAB entries.

- ① $LC = 0$
- ② While next statement is not END statement
 - (i) IF START or ORIGIN statement
 - (a) $LC = \text{value specified in operand}$

(ii) IF LTORG statement
Update LC as per the address assigned to literal in current pool.

(iii) IF declaration statement

- (a) If DC statement then assemble the constant.
- (b) $LC = LC + \text{size of operand}$

(iv) IF imperative statement

- (a) Get operand address as per entry number from SYMTAB and LITAB.
gather instructions in target code
- (b) $LC = LC + \text{size of instruction}$

③ Write target code into input file.

Consider
code g

Source

START
READ
READ
MOVER
ADD A
MOVEM
MULT A
SUB A
COMP A
LTORG
PRIN
STOP
XDS
YD
ONE
Z
A

Symbol

X

Y

O

Z

A

Consider the example which shows the target code generation from TC.

Source Program	Variant I	Target code
START 401	(AD, 00) CC, 400)	
READ X	(IS, 09) (S, 0)	09 0 412
READ Y	(IS, 109) (S, 1)	09 0 413
MOVER AREG, X	(IS, 04) 1 (S, 0)	09 1 413
ADD AREG, Y	(IS, 01) 1 (S, 1)	04 1 412
MOVEM AREG, Z	(IS, 05) 1 (S, 3)	05 1 416
MULT AREG, = 'D'	(IS, 03) 1 (L, 0)	03 1 409
SUB AREG, A	(IS, 02) 1 (S, 4)	02 1 419
COMP AREG, ONE	(IS, 06) 1 (S, 1)	06 1 415
LTOGR	(AD, 05)	
PRINT Z	(IS, 10) (S, 3)	10 0 416
STOP	(IS, 00)	00 0 000
X DS 1	(DL, 01) CC, 1)	
Y DS 2	(DL, 01) (C, 2)	
ONE DC 'D'	(DL, 02) CC, 10)	
Z DS 3	(DL, 01) (C, 3)	
A DS 1	(DL, 01) (C, 2)	
END	(AD, 02)	

Symbol Table		
Symbol	Address	Length
X	412	1
Y	413	2
ONE	415	1
Z	416	3
A	419	1

Literal Table	
Literal	Address
= 'D'	409

Pool Table	
Literal No	
#1	